DCT-Based JPEG Compression Study

Detailed Code Explanation and Analysis

Contents

1 Introduction **3**

2 Library Imports **3**
2.1 Explanation 3

3 JPEG Quantization Matrix **3**
3.1 Purpose 4
3.2 Why This Matrix Is Structured This Way 4

4 Quality-Based Quantization Scaling **4**
4.1 Purpose 4
4.2 Quality Factor 4
4.3 Why Two Different Scaling Equations? 4

5 DCT Compression Function **5**
5.1 Step 1: Image Dimensions 5
5.2 Step 2: Level Shift 5
5.3 Step 3: Block Splitting 5
5.4 Why transpose()? 6
5.5 Step 4: Forward DCT 6
5.6 DCT Formula 6
5.7 Why DCT? 6
5.8 Step 5: Quantization 6
5.9 Quantization Formula 6
5.10 Step 6: Dequantization 6
5.11 Step 7: Inverse DCT 6
5.12 Step 8: Reconstruct Image 7
5.13 Step 9: Clipping 7

6 Quality Metrics **7**
6.1 Mean Squared Error (MSE) 7
6.2 Peak Signal-to-Noise Ratio (PSNR) 7
6.3 Structural Similarity Index (SSIM) 7

7 DCT Coefficient Visualization **8**
7.1 Purpose 8
7.2 Why Use Absolute Value? 8
7.3 Why Average Across Blocks? 8
7.4 Why Log Scale? 8
7.5 Interpretation 8

8 Image Loading and Preprocessing **9**
8.1 Why Grayscale? 9

9 Compression Experiments **9**

10 Visualization of Results **9**
10.1 Expected Behavior 9

11 Graphs and Analysis **9**
11.1 Observations 10

12 Conclusion **10**

1 Introduction

This project implements a simplified JPEG-style image compression system using the Discrete Cosine Transform (DCT). The objective is to study how JPEG compression works internally by applying frequency-domain transformation, quantization, and reconstruction on grayscale images.

The project demonstrates:

- Frequency-domain image processing
- Lossy image compression
- Quantization-based information reduction
- Image quality analysis using PSNR and SSIM
- Energy compaction property of DCT

2 Library Imports

```
import cv2
import matplotlib.pyplot as plt
import numpy as np
from scipy.fftpack import dctn, idctn
from skimage.metrics import structural_similarity as ssim
```

2.1 Explanation

- **OpenCV (cv2)** is used for image loading and processing.
- **NumPy** is used for matrix operations and numerical computation.
- **Matplotlib** is used for image and graph visualization.
- **SciPy FFTPack** provides efficient implementations of DCT and inverse DCT.
- **scikit-image** provides the Structural Similarity Index (SSIM) metric.

3 JPEG Quantization Matrix

```
BASE_QUANTIZATION_MATRIX = np.array([
    [16, 11, 10, 16, 24, 40, 51, 61],
    [12, 12, 14, 19, 26, 58, 60, 55],
    [14, 13, 16, 24, 40, 57, 69, 56],
    [14, 17, 22, 29, 51, 87, 80, 62],
    [18, 22, 37, 56, 68, 109, 103, 77],
    [24, 35, 55, 64, 81, 104, 113, 92],
    [49, 64, 78, 87, 103, 121, 120, 101],
    [72, 92, 95, 98, 112, 100, 103, 99]
], dtype=np.float32)
```

3.1 Purpose

This matrix controls how aggressively each DCT frequency coefficient is compressed.

3.2 Why This Matrix Is Structured This Way

- Small values in the top-left preserve low-frequency coefficients.
- Large values in the bottom-right heavily compress high-frequency coefficients.

This works because human vision is more sensitive to low-frequency information and less sensitive to fine high-frequency details.

4 Quality-Based Quantization Scaling

```
1 def get_quantization_matrix(quality=50):
2     quality = np.clip(quality, 1, 100)
3     scale = (5000 / quality) if quality < 50 else (200 - 2 *
          quality)
4     return np.clip((BASE_QUANTIZATION_MATRIX * scale + 50) / 100,
          1, 255)
```

4.1 Purpose

This function dynamically scales the JPEG quantization matrix based on a user-selected quality factor.

4.2 Quality Factor

- Low quality value $\rightarrow$ stronger compression
- High quality value $\rightarrow$ better image quality

4.3 Why Two Different Scaling Equations?

JPEG quality scaling is nonlinear.

$$scale = \frac{5000}{Q}, \quad Q < 50$$

$$scale = 200 - 2Q, \quad Q \geq 50$$

Lower quality values produce aggressive compression while higher quality values preserve more information.

5 DCT Compression Function

```
def process_image_dct(image, quant_matrix):
    h, w = image.shape
    img_float = np.float32(image) - 128

    blocks = img_float.reshape(h // 8, 8, w // 8, 8).transpose(0,
        2, 1, 3)

    dct_blocks = dctn(blocks, axes=(-2, -1), norm='ortho')
    quantized = np.round(dct_blocks / quant_matrix)
    dequantized = quantized * quant_matrix
    recon = idctn(dequantized, axes=(-2, -1), norm='ortho')

    result = recon.transpose(0, 2, 1, 3).reshape(h, w) + 128
    return np.clip(result, 0, 255).astype(np.uint8)
```

5.1 Step 1: Image Dimensions

```
h, w = image.shape
```

Gets image height and width.

5.2 Step 2: Level Shift

```
img_float = np.float32(image) - 128
```

Pixel values are shifted from:

$$[0, 255] \rightarrow [-128, 127]$$

This centers data around zero, improving DCT efficiency.

5.3 Step 3: Block Splitting

```
blocks = img_float.reshape(h // 8, 8, w // 8, 8).transpose(0, 2,
    1, 3)
```

JPEG processes images in 8×8 blocks because:

- computationally efficient,
- good balance between quality and compression,
- minimizes visual artifacts.

5.4 Why transpose()?

The transpose reorganizes axes into:

$$(block\_row, block\_col, 8, 8)$$

allowing simultaneous vectorized block processing.

5.5 Step 4: Forward DCT

```
dct_blocks = dctn(blocks, axes=(-2, -1), norm='ortho')
```

Transforms each 8×8 block into frequency coefficients.

5.6 DCT Formula

$$F(u,v) = \frac{1}{4} C(u)C(v) \sum_{x=0}^{7} \sum_{y=0}^{7} f(x,y) \cos\left[\frac{(2x+1)u\pi}{16}\right] \cos\left[\frac{(2y+1)v\pi}{16}\right]$$

5.7 Why DCT?

DCT concentrates most image energy into low-frequency coefficients.

5.8 Step 5: Quantization

```
quantized = np.round(dct_blocks / quant_matrix)
```

This is the lossy compression step.

5.9 Quantization Formula

$$Q(u,v) = \text{round}\left(\frac{F(u,v)}{T(u,v)}\right)$$

High-frequency coefficients become zero after quantization.

5.10 Step 6: Dequantization

```
dequantized = quantized * quant_matrix
```

Approximate reconstruction of DCT coefficients.

5.11 Step 7: Inverse DCT

```
recon = idctn(dequantized, axes=(-2, -1), norm='ortho')
```

Transforms frequency coefficients back into pixel values.

5.12 Step 8: Reconstruct Image

```
result = recon.transpose(0, 2, 1, 3).reshape(h, w) + 128
```

Rebuilds the original image layout and reverses the earlier level shift.

5.13 Step 9: Clipping

```
np.clip(result, 0, 255).astype(np.uint8)
```

Ensures all pixel values remain within the valid image range.

6 Quality Metrics

```
def compute_metrics(original, compressed):
    mse = np.mean((original.astype(np.float32)
           - compressed.astype(np.float32)) ** 2)

    psnr = 100.0 if mse == 0 else \
           20 * np.log10(255 / np.sqrt(mse))

    ssim_score = ssim(original, compressed)

    return mse, psnr, ssim_score
```

6.1 Mean Squared Error (MSE)

$$MSE = \frac{1}{MN} \sum_{x=1}^{M} \sum_{y=1}^{N} [I(x,y) - K(x,y)]^2$$

Measures average pixel error.
Lower MSE indicates better reconstruction quality.

6.2 Peak Signal-to-Noise Ratio (PSNR)

$$PSNR = 20 \log_{10} \left(\frac{255}{\sqrt{MSE}} \right)$$

Measures signal quality in decibels.
Higher PSNR indicates better quality.

6.3 Structural Similarity Index (SSIM)

SSIM measures:

- luminance similarity,
- contrast similarity,

- structural similarity.

SSIM range:

$$0 \leq SSIM \leq 1$$

Higher SSIM means better perceptual quality.

7 DCT Coefficient Visualization

```
def visualize_dct_coefficients(image):
    img_float = np.float32(image) - 128

    blocks = img_float.reshape(image.shape[0] // 8,
             8,
             image.shape[1] // 8,
             8).transpose(0, 2, 1, 3)

    dct_blocks = dctn(blocks, axes=(-2, -1), norm='ortho')

    avg_coeff = np.mean(np.abs(dct_blocks), axis=(0, 1))

    plt.figure(figsize=(5, 4))
    plt.imshow(np.log(avg_coeff + 1), cmap='hot')
```

7.1 Purpose

Visualizes average DCT coefficient magnitude across all image blocks.

7.2 Why Use Absolute Value?

DCT coefficients may be positive or negative.
Magnitude represents actual energy strength.

7.3 Why Average Across Blocks?

Shows global frequency distribution across the image.

7.4 Why Log Scale?

DCT coefficients vary significantly in magnitude.
Log scaling improves visibility of smaller coefficients.

7.5 Interpretation

Bright top-left region indicates:

- strong low-frequency energy,

- dominant image structure.

Dark bottom-right region indicates:

- weak high-frequency energy,
- less important fine details.

This validates the theoretical basis of JPEG compression.

8 Image Loading and Preprocessing

```
image = cv2.imread("images/sample.jpg",
                   cv2.IMREAD_GRAYSCALE)
```

Loads grayscale image.

8.1 Why Grayscale?

Simplifies implementation and analysis.

```
image = image[:h - h % 8, :w - w % 8]
```

Ensures dimensions are divisible by 8 for block processing.

9 Compression Experiments

```
quality_levels = [5, 25, 50, 75, 95]
```

Multiple quality levels are tested to analyze the trade-off between:

- compression ratio,
- reconstruction quality.

10 Visualization of Results

Compressed images are displayed at multiple quality levels.

10.1 Expected Behavior

- Low quality $\rightarrow$ visible artifacts and high compression
- High quality $\rightarrow$ better reconstruction and larger size

11 Graphs and Analysis

PSNR and SSIM graphs visualize how image quality changes with compression level.

11.1 Observations

- PSNR increases as quality increases.
- SSIM approaches 1 at high quality.
- Low quality factors introduce blocking artifacts.

12 Conclusion

This project successfully demonstrates the core principles of JPEG image compression using the Discrete Cosine Transform.

The experiment proves that:

- DCT concentrates image energy into low frequencies.
- Quantization enables efficient lossy compression.
- High-frequency components can be removed with limited perceptual impact.
- PSNR and SSIM together provide meaningful image quality evaluation.

The implementation also demonstrates the efficiency of vectorized NumPy-based block processing compared to traditional nested loop implementations.